

Current Results Only

MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection | Updated 2026-07-09

Scope

This PDF reports the current result situation. It includes the static supervised baselines plus all three RecurrentPPO runs now present in experiments/: v1, v2 big, and RL+LLM shaped. Supervised models are still static classifiers and have not yet been adapted into ICSNADGym policies.

Key Numbers

- Best static supervised test F1: GBDT = 0.985 (accuracy 0.975, precision 0.971, recall 1.000).
- Best RecurrentPPO-family test F1 on base reward env: RL+LLM shaped = 0.909.
- Best RecurrentPPO-family test reward on base reward env: RL+LLM shaped = 2447.53.
- Base-reward comparison uses fresh val/test envs per model: episode_length=1024, seed=123, n_episodes=15.
- RL+LLM shaped was also evaluated in a shaped env; F1 is unchanged, shaped reward differs slightly due to the potential term.

Current Interpretation

Static supervised ML remains strongest as a row-level classifier. But among the RL policies, v2 improves over v1 and RL+LLM shaped is currently the strongest RecurrentPPO-family result on the base-reward test evaluation.

The full thesis comparison still needs supervised-as-policy evaluation inside ICSNADGym before claiming an operational winner.

Static Supervised Baseline Results

Current artifact: data/supervised_baseline_results.json | Not simulator-adapted

Models sorted by test F1. Metrics are static row-level classification metrics.

model	rows	valF1	testAcc	testP	testR	testF1	train_s
GBDT	300000	0.894	0.975	0.971	1.000	0.985	49.3
RandomForest	300000	0.931	0.852	0.851	1.000	0.920	3.6
AdaBoost	300000	0.936	0.848	0.851	0.995	0.917	18.9
LightGBM	300000	0.922	0.847	0.851	0.995	0.917	1.0
XGBoost	300000	0.922	0.846	0.851	0.993	0.916	6.6
ANN	300000	0.924	0.808	0.845	0.948	0.893	16.9
ELM	300000	1.000	0.712	0.827	0.835	0.831	3.9
KNN	50000	1.000	0.712	0.827	0.835	0.831	0.0
SVC	15000	1.000	0.711	0.827	0.834	0.831	0.0
GaussianNB	300000	0.898	0.553	0.788	0.647	0.711	0.1

Notes

- These models were retrained locally on the harmonized Parquet corpus and current splits; the numbers are not copied from the external paper.
- They do not yet have simulator reward, false-block cost, or quarantine behavior results.

RecurrentPPO Family: Base-Reward Evaluation

All three final models evaluated on same base ICSNADGym objective

Protocol: fresh env per model/split, episode_length=1024, seed=123, 15 episodes. LLM shaping disabled for this table.

model	steps	split	reward	std	prec	rec	f1	fp	fn
RecurrentPPO v1	500000	val	2389.67	6417.15	0.735	0.877	0.800	2572	998
RecurrentPPO v1	500000	test	-971.87	6986.12	0.932	0.706	0.803	409	2344
RecurrentPPO v2 big	2000000	val	3739.47	7793.13	0.755	0.877	0.811	2316	997
RecurrentPPO v2 big	2000000	test	1969.60	6674.06	0.996	0.734	0.845	25	2119
RL+LLM shaped	2000000	val	3808.80	7866.20	0.749	0.880	0.810	2395	972
RL+LLM shaped	2000000	test	2447.53	5143.45	0.982	0.845	0.909	120	1235

Readout

On the same base-reward simulator episodes, v2 improves over v1. RL+LLM shaped gives the strongest current RL-family test result: reward 2447.53 and F1 0.909, with high precision 0.982 and recall 0.845.

LLM-Shaped Reward and Comparison Caveat

Separating base operational reward from shaped training reward

RL+LLM shaped model evaluated with use_llm_shaping=True:

split	reward	std	prec	recall	f1
val	3749.28	7859.34	0.749	0.880	0.810
test	2387.77	5139.70	0.982	0.845	0.909

Why Earlier v1 Numbers Differ

The older Phase 3/evaluate_baseline.py script hardcodes v1 and evaluates RecurrentPPO after running always_ALLOW, always_ALERT, and oracle on the same env object. That means v1 sees a later deterministic episode sequence. The fresh-seed three-model table above is cleaner for comparing v1 vs v2 vs RL+LLM.

Legacy script v1 val: reward 2939.13, precision 0.920, recall 0.983, F1 0.950

Legacy script v1 test: reward 286.60, precision 0.684, recall 0.835, F1 0.752

Still Missing

- Supervised models still need simulator-policy adaptation for a full operational comparison.
- Recommended policy mapping: low attack probability -> ALLOW, medium -> ALERT, high -> BLOCK; tune thresholds on validation reward.